

BASIC TERMINOLOGY - ELEMENTARY DATA ORGANIZATION

Prof. Madonna Lamin

TERMINOLOGY

1. **Data** are simply values or sets of values
2. A **data item** refers to a single unit of values. Data items are divided into sub-items are called **group items**.
3. Those that we cannot divide into sub-items are called **elementary items**.

For example, an employee's name may be divided into three subitems - first name, middle name and last name but social security number or aadhar number would normally be treated as a single item.

4. Collections of data are frequently organized into hierarchy of ***fields, records and files.***

5. An ***entity*** is something that has certain attributes or properties which may be assigned values. The values themselves may be either numeric or non-numeric.

Attributes:	Name	Age	Gender	SSN
Values:	Rohland, Gail	34	F	134-24-5533

6. **Entities** with similar attributes (e.g., all employees in an organization) form an **entity set**.

7. Each attributes of an entity set has a **range of values**, the set of possible values that could be assigned to the particular attribute.

8. The term “**information**” is sometimes used for data with given attributes, or, in other words, meaningful or processed data.

9. The way that data are organized into the hierarchy of fields, records and files reflects the relationship between attributes, entities and entity sets.

- A **field** is a single elementary unit of information representing an attribute of an entity.
- A **record** is the collection of field values of a given entity.
- A **file** is the collection of records of the entities in a given entity set.

Each record in a file may contain many field items, but the value in a certain field may uniquely determine the record in the file. Such a field K is called ***primary key***, and the values $k_1, k_2, \dots$ in such a field are called keys or key values.

Example 1.1

- (a) Suppose an automobile dealership maintains an inventory file where each record contains the following data:

Serial Number, Type, Year, Price, Accessories

The Serial Number field can serve as a primary key for the file, since each automobile has a unique serial number.

- (b) Suppose an organization maintains a membership file where each record contains the following data:

Name, Address, Telephone Number, Dues Owed

Although there are four data items, Name and Address may be group items. Here the Name field is a primary key. Note that the Address and Telephone Number fields may not serve as primary keys, since some members may belong to the same family and have the same address and telephone number.

Records may also be classified according to length.

A file can have ***fixed-length records*** or ***variable-length records***

In ***fixed-length records***, all the records contain the same data items with the same amount of space assigned to each data item.

In ***variable-length records***, file records may contain different lengths.

For example, student records, usually have variable lengths, since different students take different numbers of courses.

The above organization of data into fields, records and files may not be complex enough to maintain and efficiently process certain collections of data.

For this reason, ***data are also organized into more complex type of structures***. The study of such data structure, includes the following three steps:

1. Logical or mathematical description of the structure
2. Implementation of the structure on a computer
3. Quantitative analysis of the structure, which includes determining the amount of memory needed to store the structure and the time required to process the structure.

DATA STRUCTURES

- Data may be organized in many different ways; the logical or mathematical model of a particular organization of data is called a ***data structure***.
- The choice of a particular data model depends on two consideration:
 1. It must be rich enough in structure to mirror the actual relationships of the data in real world.
 2. The structure should be simple enough that one can effectively process the data when necessary.

ARRAYS

- The simplest type of data structure is a linear (one-dimensional) array.
- By a linear array, we mean a list of a **finite number n** of **similar data elements** referenced respectively by a set of n consecutive numbers, usually $1, 2, 3, \dots, n$.
- If we choose the name A for the array, then the elements of A are denoted by subscript notation

$a_1, a_2, a_3, \dots, a_n$

Or by the parentheses notation

$A(1), A(2), A(3), \dots, A(n)$

Or by the bracket notation

$A[1], A[2], A[3], \dots, A[n]$

- Regardless of the notation, the number K in $A[K]$ is called a ***subscript*** and $A[K]$ is called a ***subscripted variable***.

Example 1.2

A linear array STUDENT consisting of the names of six students is pictured in Fig. 1.1. Here STUDENT[1] denotes John Brown, STUDENT[2] denotes Sandra Gold, and so on. STUDENT

STUDENT	
1	John Brown
2	Sandra Gold
3	Tom Jones
4	June Kelly
5	Mary Reed
6	Alan Smith

Fig. 1.1

Linear arrays are called one-dimensional arrays because each element in such an array is referenced by one subscript. A *two-dimensional array* is a collection of similar data elements where each element is referenced by two subscripts. (Such arrays are called *matrices* in mathematics, and *tables* in business applications.) Multidimensional arrays are defined analogously. Arrays will be covered in detail in Chapter 4.

Example 1.3

A chain of 28 stores, each store having 4 departments, may list its weekly sales (to the nearest dollar) as in Fig. 1.2. Such data can be stored in the computer using a two-dimensional array in which the first subscript denotes the store and the second subscript the department. If SALES is the name given to the array, then

SALES[1, 1] = 2872, SALES[1, 2] = 805, SALES[1, 3] = 3211, ..., SALES[28, 4] = 982

Dept. Store	1	2	3	4
1	2872	805	3211	1560
2	2196	1223	2525	1744
3	3257	1017	3686	1951
...	...	...	...	...
28	2618	931	2333	982

Fig. 1.2

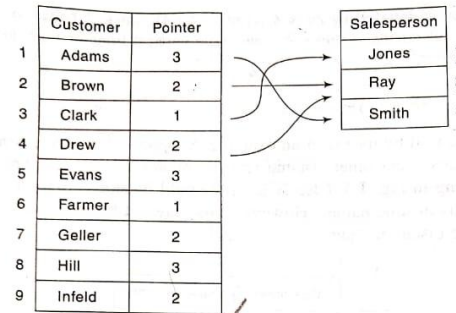
The size of the array is denoted by 28 X 4, since it contains 28 rows and 4 columns.

LINKED LISTS

Suppose a brokerage firm maintains a file where record contains a customer's name and his or her salesperson, and suppose the file contains the data appearing in the table shown below.

	Customer	Salesperson
1	Adams	Smith
2	Brown	Ray
3	Clark	Jones
4	Drew	Ray
5	Evans	Smith
6	Farmer	Jones
7	Geller	Ray
8	Hill	Smith
9	Infeld	Ray

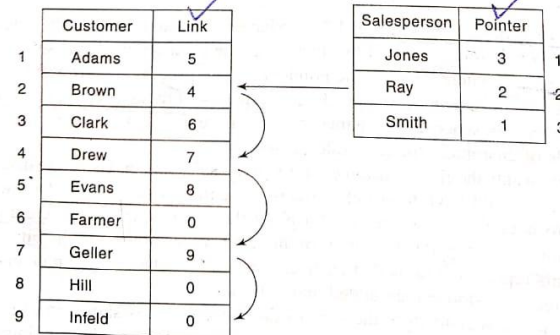
- Another way of storing the data above is to have a separated array for the salespeople and an entry (called a pointer) in the customer file which gives the location of each customer's salesperson. Fig 1.4
- Fig 1.5 - Each salesperson have a set of pointers giving the positions of his or her customers.
- Fig 1.6 - Each sales person has one pointer which points to his or her first customer, whose pointer in turn points to the second customer, and so on, with the salesperson's last customer indicate by 0. This is an example of linked list.



✓ Fig. 1.4

	Salesperson	Pointer
1	Jones	3, 6
2	Ray	2, 4, 7, 9
3	Smith	1, 5, 8

✓ Fig. 1.5



✓ Fig. 1.6

TREES

- Data often contain a hierarchical relationship between various elements. The data structure that reflects this relationship is called a rooted tree graph or simply a tree.

Example 1.4 Record Structure

Although a file may be maintained by means of one or more arrays, a record, where one indicates both the group items and the elementary items, can best be described by means of a tree structure. For example, an employee personnel record may contain the following data items:

✓ Social Security Number, Name, Address, Age, Salary, Dependents

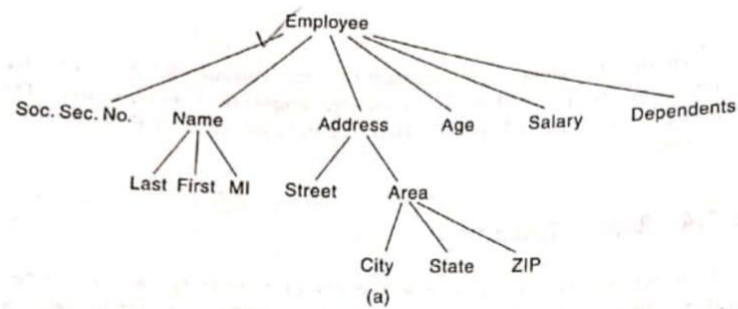
However, Name may be a group item with the subitems Last, First and MI (middle initial). Also, Address may be a group item with the subitems Street address and Area address, where Area itself may be a group item having subitems City, State and ZIP code number. This hierarchical structure is pictured in Fig. 1.7(a). Another way of picturing such a tree structure is in terms of levels, as in Fig. 1.7(b).

Example 1.5 Algebraic Expressions

Consider the algebraic expression

✓
$$(2x + y)(a - 7b)^3$$

Using a vertical arrow ($\uparrow$) for exponentiation and an asterisk (*) for multiplication, we can represent the expression by the tree in Fig. 1.8. Observe that the order in which the operations will be performed is reflected in the diagram: the exponentiation must take place after the subtraction, and the multiplication at the top of the tree must be executed last.



```

01 Employee
  02 Social Security Number
  02 Name
    03 Last
    03 First
    03 Middle Initial
  02 Address
    03 Street
    03 Area
      04 City
      04 State
      04 ZIP
  02 Age
  02 Salary
  02 Dependents
  
```

(b)

Fig. 1.7

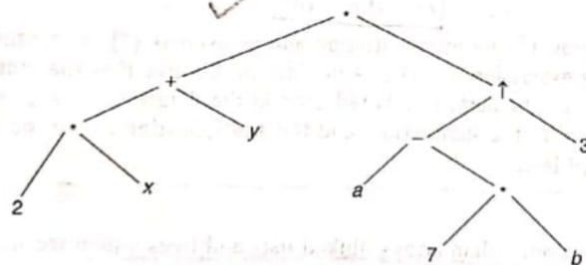


Fig. 1.8

There are data structures other than arrays, linked list and trees. Some of these structures are briefly described below:

1. Stack:

A stack also called a ***last-in first-out (LIFO)*** system, is a linear list in which insertions and deletions can take place only at one end, called the top. This structure is similar in its operation to a stack of dishes on a spring system as shown in the figure. Note that new dishes are inserted only at the top of the stack and dishes can be deleted only from the top of the stack

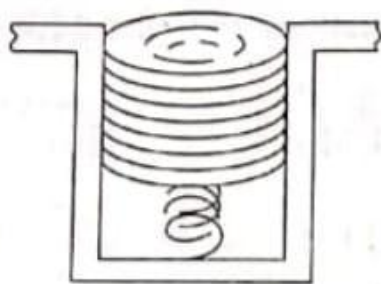
2. *Queue*:

A queue, also called a ***first-in first-out (FIFO) system*** is a linear list in which deletions can take place only at one end of the list “the front” of the list, and insertions can take place only at the other end of the list, “the rear” of the list. This structure operates much the same way as a line of people waiting at a bus stop: the first person in the line is the first person to board the bus.

3. ***Graph:***

Data sometimes contain a relationship between pairs of elements which is not necessarily hierarchical in nature.

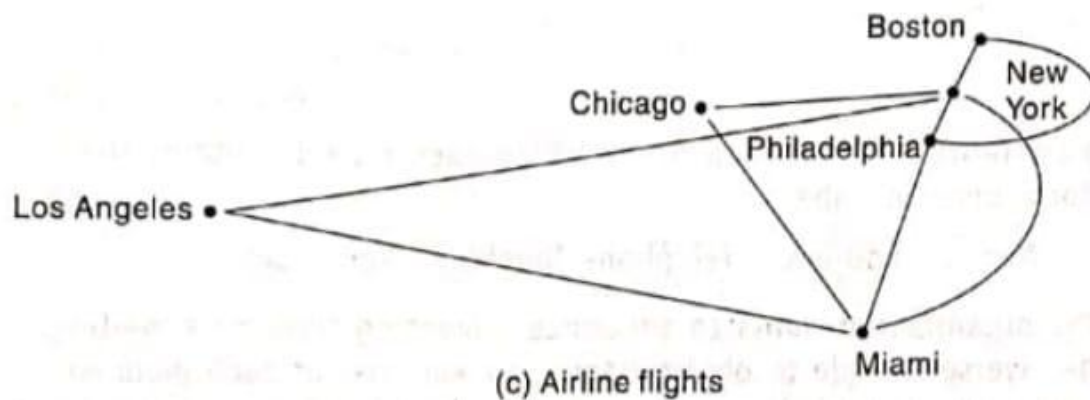
For example, suppose an airline flies only between cities connected by lines in the figure. The data structure which reflects this type of relationship is called a Graph.



(a) Stack of dishes



(b) Queue waiting for a bus



(c) Airline flights

Fig. 1.9

DATA STRUCTURE OPERATIONS

- The data appearing in our data structure are processed by means of certain operation. In fact, the particular data structure that one chooses for a given situation depends largely on the frequency with which specific operations are performed.
- The following four operations play a major role for this course:
 - **Traversing:** Accessing each record exactly once so that certain items in the record may be processed. This accessing and processing is sometimes called “visiting” the record

DATA STRUCTURE OPERATIONS

- **Searching:** Finding the location of the record with a given key value or finding the location of all records which satisfy one or more conditions
- **Inserting:** Adding a new record to the structure
- **Deleting:** Removing a record from the structure

The following two operations, which are used in special situations :

- **Sorting:** Arranging the records in some logical order (e.g., alphabetically according to some NAME key, or in numerical order according to some NUMBER key, such as Social Security Number or account number)
- **Merging:** Combining the records in two different sorted files into a single sorted file.

Other operations like **copying** or **concatenation** will be discussed later.

Example 1.6

✓ An organization contains a membership file in which each record contains the following data for a given member:

Name, Address, Telephone Number, Age, Sex

- (a) Suppose the organization wants to announce a meeting through a mailing. Then one would traverse the file to obtain Name and Address for each member.
- (b) Suppose one wants to find the names of all members living in a certain area. Again one would traverse the file to obtain the data.
- (c) Suppose one wants to obtain Address for a given Name. Then one would search the file for the record containing Name.
- (d) Suppose a new person joins the organization. Then one would insert his or her record into the file.
- (e) Suppose a member dies. Then one would delete his or her record from the file.
- (f) Suppose a member has moved and has a new address and telephone number. Given the name of the member, one would first need to search for the record in the file. Then one would perform the "update"—i.e., change items in the record with the new data.
- (g) Suppose one wants to find the number of members 65 or older. Again one would traverse the file, counting such members.

ALGORITHMS : COMPLEXITY, TIME SPACE TRADEOFF

- An **algorithm** is a well-defined list of steps for solving a particular problem. One major purpose of this course, is to develop efficient algorithms for the processing of our data.
- The **time and space** it uses are two major measures of the efficiency of an algorithm.
- The complexity of an algorithm is the **function** which gives the running time and/or space in terms of the input size.

SEARCHING ALGORITHMS

Consider a membership file, as in Example 1.6, in which each record contains, among other data the name and telephone number of its members. Suppose we are given the name of a member and we want to find his or her telephone number. One way to do is to linearly search through the file that is to apply the following algorithm.

LINEAR SEARCH

Search each record of the file one at a time, until finding the given Name and hence the corresponding Telephone Number.

- The time required to execute the algorithm is proportional to the number of comparisons. Also, assuming that each name in the file is equally likely to be picked., it is intuitively clear that the average number of comparisons for a file with n records is equal to $n/2$ that is the complexity of the linear search algorithm is $C(n) = n/2$
- The above algorithm would be impossible in practice if we were searching through a list consisting of thousands of names, as in a telephone book..

BINARY SEARCH

- Compare the given name Name with the name in the middle of the list; this tells which half of the list contains Name.
- Then compare Name with the name in the middle of the correct half to determine which quarter of the list contains Name.
- Continue this process until finding Name in the list
- The complexity of the binary search algorithm is given by:

$$C(n) = \log n$$

- For example, one will not require more than 15 comparisons to find a given Name in a list containing 25000 names.

- Although, the binary search algorithm is very efficient algorithm, it has some major drawbacks.
 - Specifically, the algorithm assumes that one has direct access to the middle name in the list or a sublist. This means that the list must be stored in some type of array.

AN EXAMPLE OF TIME-SPACE TRADEOFF

- Suppose a file of records contains names, social security number and much additional information among its fields.
- Sorting the file alphabetically and using a binary search is a very efficient way to find the record for a given name.
- On the other hand, suppose we are given only the social security number of the person. Then we have to do linear search for the record, which is extremely time-consuming for a very large number of records.

- Have another file which is sorted numerically according to social security number. This however, would double the space required for storing the data.
- Another way, as shown in the figure, is to have the main file sorted numerically by social security number and to have an auxiliary array with only two columns, the first column containing an alphabetized list of the names and the second column containing pointers which give the locations of the corresponding records in the main file.

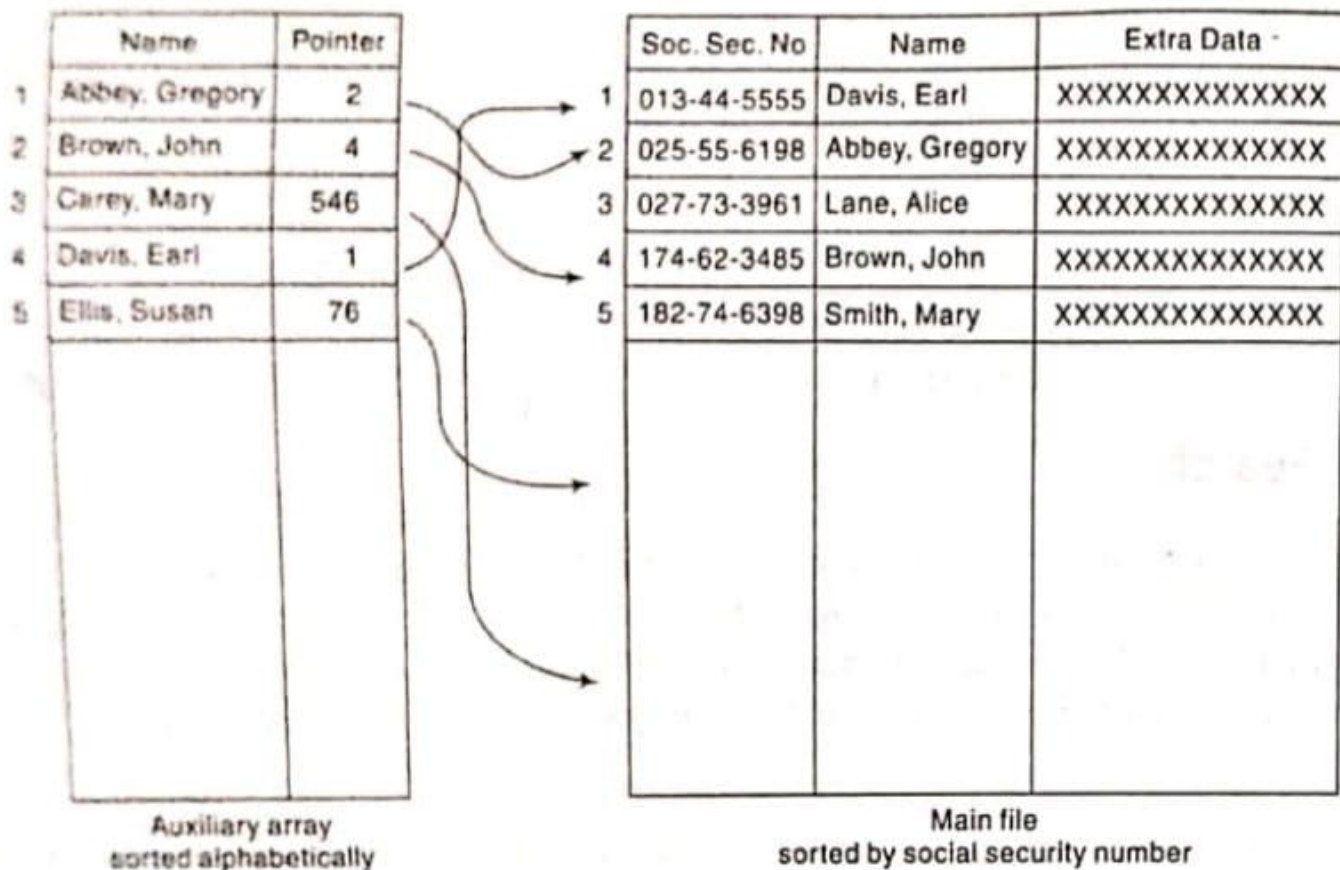


Fig. 1.10

EXERCISES

1. Consider the linear array NAME in the figure. The values of FIRST and LINK[K] in the figure determines a linear ordering of the names as follows. FIRST gives the location of the first name in the list, and LINK[K] gives the location of the name following NAME[K], with 0 denoting the end of the list. Find the linear ordering of the names.

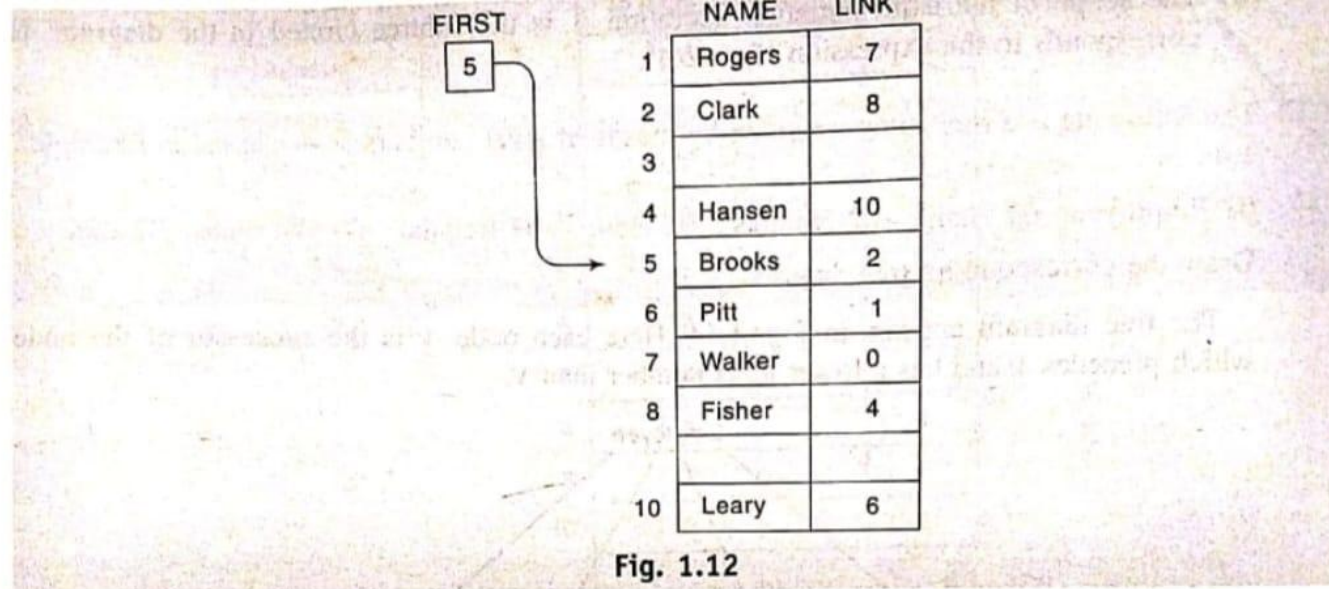


Fig. 1.12

The ordering is obtained as follows:

FIRST = 5, so the first name in the list is NAME[5], which is Brooks.

LINK[5] = 2, so the next name is NAME[2], which is Clark.

LINK[2] = 8, so the next name is NAME[8], which is Fisher.

LINK[8] = 4, so the next name is NAME[4], which is Hansen.

LINK[4] = 10, so the next name is NAME[10], which is Leary.

LINK[10] = 6, so the next name is NAME[6], which is Pitt.

LINK[6] = 1, so the next name is NAME[1], which is Rogers.

LINK[1] = 7, so the next name is NAME[7], which is Walker.

LINK[7] = 0, which indicates the end of the list.

Thus the linear ordering of the names is Brooks, Clark, Fisher, Hansen, Leary, Pitt, Rogers, Walker. Note that this is the alphabetical ordering of the names.

2. Consider the algebraic expression $(7x + y)(5a - b)^3$.

- a. Draw the corresponding tree diagram
- b. Find the scope of the exponential operation (The scope of a node v in a tree is the subtree consisting of v and the nodes following v)

3. Discuss whether a stack or a queue is the appropriate structure for determining the order in which elements are processed in each of the following situations:

1. Batch computer programs are submitted to the computer center.
2. Program A calls subprogram B, which calls subprogram C, and so on.
3. Employees have a contract which calls for a seniority system for hiring and firing.

Answer

1. Queue. Excluding priority cases, programs are executed on a first come , first served basis
2. Stack. The last subprogram is executed first, and its results are transferred to the next-to-last program, which is then executed, and so on, until the original calling program is executed.
3. Stack. In a seniority system, the last to be hired is the first to be discharged.

4. Briefly describe the notions of (a) the complexity of an algorithm (b) the space-time tradeoff of algorithms.

Answer:

- (a) The complexity of an algorithm is a function $f(n)$ which measures the time and/or space used by an algorithm in terms of the input size n .
- (b) The space-time tradeoff refers to a choice between algorithmic solutions of a data processing problem that allows one to decrease the running time of an algorithmic solution by increasing the space to store the data and vice versa.

5. Suppose a data set S contains n elements.
- a) Compare the running time T_1 of the linear search algorithm with the running time T_2 of the binary search algorithm when (i) $n = 1000$ and (ii) $n = 10,000$
 - b) Discuss searching for a given item in S when S is stored as a linked list

Answer:

- a) The expected running time of the linear search algorithm is $f(n) = n/2$ and that of the binary search is $f(n) = \log n$.
 - i) for $n = 1000$, $T1 = 500$ and $T2 = 10$
 - ii) for $n = 10,000$, $T1 = 5000$ and $T2 = 14$
- b) The binary search algorithm assumes that one can directly access the middle element in the set S . But one cannot directly access the middle element in a linked list. Hence one may have to use a linear search algorithm where S is stored as a linked list.

6. The daily flights of an airline company appear in the following figure. CITY lists the cities and ORIG[K] and DEST[K] denote the cities of origin and destination, respectively, of the flight NUMBER[k]. Draw the corresponding directed graph of the data. (The graph is directed because the flight numbers represent flights from one city to another but not returning.

	CITY
1	Atlanta
2	Boston
3	Chicago
4	Miami
5	Philadelphia

(a)

	NUMBER	ORIG	DEST
1	701	2	3
2	702	3	2
3	705	5	3
4	708	3	4
5	711	2	5
6	712	5	2
7	713	5	1
8	715	1	4
9	717	5	4
10	718	4	5

(b)

Fig. 1.15

7. Consider the data above, which gives the different flights of an airline. Discuss different ways of storing the data so as to decrease the time in executing the following:

- a) Find the origin and destination of a flight, given the flight number.
- b) Give city A and city B, find whether there is a flight from A to B, and if there is find its flight number.

Answer:

a) Store the data in arrays ORIG and DEST where the subscript is the flight number.

b) Store the data in a two-dimensional array FLIGHT where FLIGHT[J,K] contains the flight number of the flight from CITY[J] to CITY[K], or contains 0 when there is no such flight.

	ORIG	DEST
701	2	3
702	3	2
703	0	0
704	0	0
705	5	3
706	0	0
⋮	⋮	⋮
715	1	4
716	0	0
717	5	4
718	4	5

(a)

FLIGHT	1	2	3	4	5
1	0	0	0	715	0
2	0	0	701	0	711
3	0	702	0	708	0
4	0	0	0	0	718
5	713	712	705	717	0

(b)

Fig. 1.17

MATHEMATICAL NOTATIONS AND FUNCTIONS

The various mathematical functions which appear very often in the analysis of algorithms and in computer science, and their notations.

FLOOR AND CEILING FUNCTIONS

Floor and Ceiling Functions

Let x be any real number. Then x lies between two integers called the floor and the ceiling of x . Specifically,

$\lfloor x \rfloor$, called the *floor* of x , denotes the greatest integer that does not exceed x .

$\lceil x \rceil$, called the *ceiling* of x , denotes the least integer that is not less than x .

If x is itself an integer, then $\lfloor x \rfloor = \lceil x \rceil$; otherwise $\lfloor x \rfloor + 1 = \lceil x \rceil$.

Example 2.1

$$\lfloor 3.14 \rfloor = 3, \quad \lfloor \sqrt{5} \rfloor = 2, \quad \lfloor -8.5 \rfloor = -9, \quad \lfloor 7 \rfloor = 7$$

$$\lceil 3.14 \rceil = 4, \quad \lceil \sqrt{5} \rceil = 3, \quad \lceil -8.5 \rceil = -8, \quad \lceil 7 \rceil = 7$$

REMAINDER FUNCTION; MODULAR ARITHMETIC

- The **remainder function**, commonly referred to in the context of modular arithmetic, plays a fundamental role in computer science, cryptography, algorithms, and number theory. Let's break it down clearly.
- In mathematics and computer science, the remainder function refers to the operation that returns the remainder after one integer is divided by another.
- This is typically denoted using the modulo operator, written as:

$a \bmod n$

Example:

$$17 \bmod 5 = 2$$

Because $17 \div 5 = 3$ with a remainder of 2.

Modular arithmetic is a system of arithmetic for integers where numbers "wrap around" upon reaching a certain value – the modulus.

Think of a clock: after 12 hours, it wraps back to 1. This is "mod 12" arithmetic.

$14 \bmod 12 = 2 \rightarrow 14 \text{ o'clock is } 2 \text{ PM.}$

In general,

For integers a and n (with $n > 0$),

$a \bmod n = r$ means that $a = q \times n + r$, where:

- q is the quotient (integer division),
- r is the remainder,
- and $0 \leq r < n$.

HOW IS MODULAR ARITHMETIC USED IN COMPUTER SCIENCE?

Modular arithmetic is everywhere in computer science. Here are key applications:

1. Hashing and Hash Tables

Used to map data of arbitrary size to fixed-size values.

Example: If a hash table has 10 slots, use $\text{hash}(\text{key}) \bmod 10$ to determine which slot to store the data in.

Ensures the index stays within valid bounds.

2. Cryptography

Many encryption algorithms (like RSA, Diffie-Hellman) rely heavily on modular arithmetic.

Modular exponentiation: computing $(a^b) \bmod n$ efficiently.

Security relies on the difficulty of problems like modular discrete logarithms or factoring large numbers.

3. Random Number Generation

Pseudo-random number generators (PRNGs) often use modular operations.

Linear Congruential Generator (LCG):

$$X_{\{n+1\}} = (a \times X_n + c) \bmod m$$

4. Circular Buffers and Arrays

Used in data structures like queues or ring buffers.

Index calculation: **$(\text{index} + 1) \bmod \text{buffer_size}$** wraps around to 0 when reaching the end.

5. Even/Odd, Parity, and Divisibility Checks

$n \bmod 2 == 0 \rightarrow$ even number

$n \bmod 3 == 0 \rightarrow$ divisible by 3

Used in algorithms to make decisions based on divisibility.

6. Computer Arithmetic (Fixed-Size Integers)

Computers use modular arithmetic naturally due to fixed bit widths.

An 8-bit unsigned integer can represent 0 to 255.

Adding 1 to 255 gives 0 $\rightarrow$ this is addition modulo 256.

IMPORTANT PROPERTIES OF MODULO ARITHMETIC

Let a , b , n be integers, and $n > 0$. Then:

1. $(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$
2. $(a - b) \bmod n = [(a \bmod n) - (b \bmod n)] \bmod n$
3. $(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n$
4. $a \bmod n = b \bmod n \rightarrow a \equiv b \pmod{n}$ (congruence)

These allow efficient computation, especially with large numbers.

Handling Negative Numbers:

In programming, the behavior of modulo with negative numbers can vary.

Mathematical definition: remainder should be non-negative.

Example: $-7 \bmod 5 = 3$ because $-7 = (-2) \times 5 + 3$

But in some languages (like Python), $-7 \% 5$ gives 3, while in C/C++, it may give -2.

So, be cautious – always check how your programming language handles negative modulo.

SUMMARY

Concept	Explanation
Modulo operation	Returns the remainder of division: $a \bmod n$
Core Idea	Numbers wrap around after reaching modulus n
Notation	$a \bmod n$ or $a \% n$
Key Uses	Hashing, Cryptography, Circular buffers
Critical insights	Enables bounded, cyclic and secure computations

EXERCISE - 1

Compute the following modulo operations. Assume positive integers unless stated.

1. $15 \bmod 4$
2. $100 \bmod 7$
3. $23 \bmod 5$
4. $0 \bmod 9$
5. $64 \bmod 8$

ANSWER EXERCISE-1

$$15 \bmod 4 = 3$$

$$100 \bmod 7 = 2 \quad (7 \times 14 = 98, \quad 100 - 98 = 2)$$

$$23 \bmod 5 = 3$$

$$0 \bmod 9 = 0$$

$$64 \bmod 8 = 0$$

EXERCISE-2

Use modular arithmetic properties to simplify and compute:

1. $(17 + 25) \bmod 6$

→ Compute using: $(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$

2. $(13 \times 8) \bmod 5$

→ Use: $(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n$

ANSWER EXERCISE-2

$$1. (17+25)=42 \rightarrow 42 \bmod 6 = 0$$

$$\text{Also: } 17 \bmod 6 = 5, 25 \bmod 6 = 1, (5+1) \bmod 6 = 0$$

$$2. 13 \bmod 5 = 3, 8 \bmod 5 = 3, 3 \times 3 = 9 \rightarrow 9 \bmod 5 = 4$$

$$\text{Direct: } 13 \times 8 = 104 \rightarrow 104 \bmod 5 = 4$$

EXERCISE-3

In a program, you want to trigger an action every 5th iteration in a loop from $i = 1$ to 30.

Write the condition using modulo to detect every 5th iteration.

Answer:

```
if i % 5 == 0: (triggers at i=5,10,15,...)
```

EXERCISE -4

A hash table has 13 slots. Given keys with hash values:

1. 100 → which slot?
2. 237 → which slot?
3. 1000 → which slot?

(Use $\text{hash}(\text{key}) \bmod 13$)

Answer:

$$100 \bmod 13 = 9 \quad (13 \times 7 = 91, \quad 100 - 91 = 9)$$

$$237 \bmod 13: \quad 13 \times 18 = 234 \rightarrow 3$$

$$1000 \bmod 13: \quad \text{Try } 13 \times 76 = 988 \rightarrow 12$$

EXERCISE-5

You're implementing a circular buffer of size 8.

Current index = 7.

After adding 3 elements (incrementing index each time), what is the new index?

→ Use: $(\text{index} + 1) \bmod 8$ repeatedly.

Answer:

$(7+1) \bmod 8 = 0$, then 1, then 2 → After 3 steps: index = 2

EXERCISE-6

An 8-bit register stores unsigned integers.

What is the result of $250 + 10$ in this register?

(Hint: It wraps around using mod 256)

Answer:

$$250 + 10 = 260 \rightarrow 260 \bmod 256 = 4$$

INTEGER AND ABSOLUTE VALUE FUNCTIONS

- **Integer functions** deal with integers - whole numbers (positive, negative and zero), without fractional parts. In programming and computation, integers are often handled differently from real (floating-point) numbers due to efficiency and precision.
- Common integer related functions

1. Floor Function – $\lfloor x \rfloor$ – The greatest integer less than or equal to x

Notation: `floor(x)` or $\lfloor x \rfloor$

Examples:

$$\lfloor 3.7 \rfloor = 3$$

$$\lfloor -2.3 \rfloor = -3$$

$$\lfloor 5 \rfloor = 5$$

- Use in computer science
 - Array indexing: converting floating-point results to valid indices.
 - Algorithms involving division and remainders.
 - Binary search, tree height calculations.

2. Ceiling Function – $\lceil x \rceil$

Definition: The smallest integer greater than or equal to x .

Notation: $\text{ceil}(x)$ or $\lceil x \rceil$

Examples:

$$\lceil 3.2 \rceil = 4$$

$$\lceil -1.7 \rceil = -1$$

$$\lceil 6 \rceil = 6$$

Use in CS:

- Memory allocation: rounding up to next whole block.
- Pagination: number of pages = $\lceil \text{total_items} / \text{items_per_page} \rceil$
- Network packet sizing.

3. Truncate (or Truncation)

Definition: Remove the fractional part of a number – toward zero.

Examples:

`trunc(4.9) = 4`

`trunc(-4.9) = -4`

Difference from floor:

For negative numbers, `trunc(-4.9) = -4`, but `floor(-4.9) = -5`.

Use in CS:

- Type casting: `(int)3.14` in C/C++ truncated to 3.
- Fast conversion when rounding toward zero is acceptable.

4. Round Function

Definition: Round to the nearest integer. Midpoints (like 2.5) usually round to the nearest even integer (banker's rounding), though behavior varies.

Examples:

`round(2.3) = 2`

`round(2.7) = 3`

`round(2.5) = 2` (in many systems)

Use in CS:

- User-facing output (e.g., display numbers).
- Converting sensor data or floating-point results to integers.

Absolute Value Function – $|x|$ or `abs(x)`

Definition

Returns the magnitude (distance from zero) of a number, ignoring the sign.

Mathematical notation: $|x|$

Function:

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{if } x < 0 \end{cases}$$

Examples:

$$|5| = 5$$

$$|-8| = 8$$

$$|0| = 0$$

Applications of absolute function in computer science:

1. Distance and error measurement

Compute distance between two points on a line: $|a - b|$

Measure error or deviation:

$|\text{predicted} - \text{actual}| \rightarrow \text{absolute error}$

Applications of absolute function in computer science:

2. Numerical Stability

Avoid negative values when only magnitude matters.

Prevent invalid inputs (e.g., negative array size).

SUMMARY

Function	Purpose	Example	CS use case
<code>floor(x)</code>	Greatest integer $\leq x$	$\lfloor 3.9 \rfloor = 3$	Array indexing
<code>ceil(x)</code>	Smallest integer $\geq x$	$\lceil 3.1 \rceil = 4$	Memory allocation
<code>trunc(x)</code>	Remove decimal part	<code>trunc(-4.8) = -4</code>	Type casting
<code>round(x)</code>	Nearest integer	<code>round(2.6) = 3</code>	Data display
<code>abs(x)</code>	Magnitude (no sign)	<code>abs(-7) = 7</code>	Distance, error

SUMMATION SYMBOLS; SUMS

- In algorithm analysis, we often need to compute how many times a statement runs – especially inside loops. Summation (denoted by the Greek letter sigma: Σ) lets us express and simplify these repeated operations.

- General Notation:

$$\sum_{i=a}^b f(i)$$

It means add up $f(i)$ for all integer values of i from a to b

COMMON SUMMATIONS AND CLOSED FORMULAS

1. Sum of 1s (Constant sum):

$$\sum_{i=1}^n 1 = n$$

It means you're doing a constant operation n times.

Use case:

A simple loop that runs n times:

```
for(i = 0; i < n; i++){  
    print("Hello there...")  
}
```

COMMON SUMMATIONS AND CLOSED FORMULAS

1. Sum of first n integers:

$$\sum_{i=1}^n i = 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2)$$

Use Case: Nested loops where inner loop grows with i .

```
for (i = 0; i < n; i++){  
    for(j = 0; j <= i; j++){  
        print(i,j)  
    }  
}
```

COMMON SUMMATIONS AND CLOSED FORMULAS

3. Sum of first n odd numbers:

$$\sum_{i=1}^n (2i - 1) = 1 + 3 + 5 + \cdots + (2n - 1) = n^2$$

Use Case: Algorithms that process odd indices or have quadratic behavior.

COMMON SUMMATIONS AND CLOSED FORMULAS

4. Sum of squares:

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

Use Case: Triple nested loops or polynomial-time algorithms.

```
for (i = 0; i < n; i++){  
    for(j = 0 ; j < i; j++){  
        for(k = 0; k < j; k++){  
            #some O(1) operations  
        }  
    }  
}
```

COMMON SUMMATIONS AND CLOSED FORMULAS

5. Sum of cubes:

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2 = \Theta(n^4)$$

Less common, but appears in higher-order algorithms.

Note: Sum of cubes = square of sum of integers.

COMMON SUMMATIONS AND CLOSED FORMULAS

6. Arithmetic Series

$$\sum_{i=0}^{n-1} (a + id) = \frac{n}{2} \cdot [2a + (n-1)d]$$

General form of linear sequences.

Example: Loop with step size or offset.

COMMON SUMMATIONS AND CLOSED FORMULAS

7. Geometric Series

Finite Geometric Sum:

$$\sum_{i=0}^k r^i = 1 + r + r^2 + \dots + r^k = \frac{r^{k+1} - 1}{r - 1} \quad (\text{for } r \neq 1)$$

Common Case: $r = 2$

$$\sum_{i=0}^k 2^i = 2^{k+1} - 1$$

Infinite Geometric Series (if $|r| < 1$):

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1 - r}$$

Use Case:

- Binary tree levels: number of nodes at each level is 2^i
- Recursion trees: work per level decreases geometrically.

COMMON SUMMATIONS AND CLOSED FORMULAS

8. Harmonic Series

$$\sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \approx \ln n + \gamma = \Theta(\log n)$$

Where γ is the Euler-Mascheroni constant (~ 0.5772).

Use Case:

- Average case of hash table insertion with chaining.
- Randomized algorithms like quicksort analysis.
- Coupon collector problem: Expected time to collect all n coupons is $n \cdot H_n \approx n \ln n$

SUMMATION	CLOSED FORM	ASYMPTOTIC GROWTH
$\sum_{i=1}^n 1$	n	$\Theta(n)$
$\sum_{i=1}^n i$	$\frac{n(n+1)}{2}$	$\Theta(n^2)$
$\sum_{i=1}^n i^2$	$\frac{n(n+1)(2n+1)}{6}$	$\Theta(n^3)$
$\sum_{i=1}^n i^3$	$\left(\frac{n(n+1)}{2}\right)^2$	$\Theta(n^4)$
$\sum_{i=0}^k 2^i$	$2^{k+1} - 1$	$\Theta(2^n)$ if $k = n$
$\sum_{i=1}^n \frac{1}{i}$	$H_n \approx \ln n$	$\Theta(\log n)$

FACTORIAL FUNCTION

- The factorial function is a fundamental mathematical concept widely used in computer science, combinatorics, probability, algorithm analysis, and recursion.
- What is the factorial function?
 - The factorial of a non-negative integer n , denoted by $n!$, is the product of all positive integers less than or equal to n .

 **Definition:**

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 2 \times 1$$

And by definition:

$$0! = 1 \quad (\text{yes, zero factorial is } 1!)$$

N	$N!$	CALCULATION
0	$0! = 1$	(by definition)
1	$1! = 1$	
2	$2! = 2 \times 1 = 2$	
3	$3! = 3 \times 2 \times 1 = 6$	
4	$4! = 4 \times 3 \times 2 \times 1 = 24$	
5	$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$	
6	$6! = 720$	



Recursive Definition:

The factorial function is a classic example of recursion:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n - 1)! & \text{if } n > 0 \end{cases}$$

Example:

$$4! = 4 \times 3! = 4 \times 3 \times 2! = 4 \times 3 \times 2 \times 1! = 4 \times 3 \times 2 \times 1 \times 0! = 24$$

WHERE IS FACTORIAL USED IN COMPUTER SCIENCE?

1. Combinatorics & Permutations

- Number of ways to arrange n distinct items = $n!$
- Example: 3 books $\rightarrow 3!=6$ possible orders.
- Number of permutations of r items from n :

- Number of permutations of r items from n :

$$P(n, r) = \frac{n!}{(n-r)!}$$

- Number of combinations (order doesn't matter):

$$C(n, r) = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Used in algorithms involving **search**, **backtracking**, and **generating all possibilities**.

2. Algorithm Analysis

- **Time complexity** of algorithms that generate all permutations: $O(n!)$
 - Example: Brute-force solution to the **Traveling Salesman Problem (TSP)** checks all $n!$ routes.
- $n!$ grows faster than 2^n , so $O(n!)$ algorithms are **intractable** for large n .

N	$N!$	TIME AT 1M OPS/SEC
5	120	0.12 milliseconds
10	3.6 million	~3.6 seconds
15	~1.3 trillion	~15 days
20	$\sim 2.4 \times 10^{18}$	~77,000 years

PROPERTIES OF FACTORIAL

PROPERTY	EXPLANATION
$0! = 1$	Base case; consistent with empty product
$n! = n \times (n - 1)!$	Recursive definition
$n!$ grows faster than a^n	For any constant a
$\log(n!) = \Theta(n \log n)$	Useful in complexity analysis
Not defined for negative or non-integers	But gamma function extends it: $\Gamma(n + 1) = n!$

PERMUTATIONS

A permutation is an arrangement of all or part of a set of objects in a specific order.

Unlike combinations, order matters in permutations.

📌 Formal Definition:

A permutation of a set is any **reordering** of its elements.

For a set with n distinct elements:

- The number of **total permutations** is $n!$ (n factorial).
- The number of **r -permutations** (arrangements of r items from n) is:

$$P(n, r) = \frac{n!}{(n - r)!}$$

✓ Examples:

1. Full Permutations of {A, B, C}:

There are $3! = 6$ permutations:

1 ABC, ACB, BAC, BCA, CAB, CBA

2. 2-permutations of {X, Y, Z}:

$$P(3, 2) = \frac{3!}{(3-2)!} = \frac{6}{1} = 6$$

1 XY, XZ, YX, YZ, ZX, ZY

"▲ Note: $XY \neq YX \rightarrow$ order matters! "

◆ Why Are Permutations Important in Computer Science?

✓ 1. Algorithm Design & Analysis

- **Backtracking algorithms** generate permutations to explore all possible solutions.
- Example: **Traveling Salesman Problem (TSP)** — find the shortest route visiting all cities → check all $n!$ permutations (brute-force).
- Time complexity: $O(n!)$ — very slow for large n .

✓ 2. Sorting and Searching

- Sorting can be viewed as finding the **correct permutation** that arranges elements in order.
- Lower bound for comparison-based sorting: $\Omega(n \log n)$, derived from $\log_2(n!) \approx n \log n$ (using Stirling's approximation).



✓ 3. Cryptography

- Permutations are used in **ciphers** (e.g., substitution ciphers, block ciphers like AES).
- Shuffling data using permutation patterns enhances security.

✓ 4. Artificial Intelligence & Search

- In **constraint satisfaction problems (CSPs)** or **scheduling**, you may need to try different orderings (permutations) of tasks.
- Used in **genetic algorithms** and **heuristics**.

✓ 5. Randomization & Sampling

- **Shuffling** a list (e.g., music playlist, cards in a game) uses random permutations.
- **Fisher-Yates shuffle** is a standard algorithm to generate a uniformly random permutation.

EXPONENTS AND LOGARITHMS

- When analyzing algorithms, especially in computer science and data structures, we often need to describe how the runtime or space requirements of an algorithm grows as the input size increases.
- Two mathematical concepts that are essential in this analysis are exponents and logarithms. They help us characterize the efficiency of algorithms and compare their performance using Big O notation.

WHY EXPONENTS AND LOGARITHMS MATTER?

- Algorithms can have vastly different performance characteristics. Some run very quickly even on large inputs, while others become impractical as input size grows. Exponents and logarithms allow us to categorize these behaviors into meaningful growth rates:
- **Exponential growth** (e.g., 2^n) → very fast growth → inefficient for large inputs
- **Logarithmic growth** (e.g., $\log n$) → very slow growth → highly efficient

Understanding these helps us choose the right algorithm for the job.

EXPONENTS IN ALGORITHM ANALYSIS

An **exponent** represents repeated multiplication: $a^n = a \times a \times \cdots \times a$ (n times). In algorithm analysis, exponential time complexity arises when the number of operations grows exponentially with input size.

Example: Exponential Time $O(2^n)$

Consider a recursive algorithm to compute Fibonacci numbers naively:

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

This function makes two recursive calls for each input, leading to a call tree that doubles at each level. The number of calls grows like 2^n , which is exponential.

Implication: Even for moderate values of n (e.g., $n=50$), the algorithm becomes extremely slow.

Use Case: Exponential algorithms are typically only feasible for very small inputs.

LOGARITHMS IN ALGORITHM ANALYSIS

A **logarithm** is the inverse of exponentiation. If $b^y = x$, then $\log_b x = y$. In algorithm analysis, logarithms usually appear with base 2 (binary logarithm, $\log_2 n$), because many algorithms divide the problem in half at each step.

Example: Logarithmic Time $O(\log n)$

BINARY SEARCH IS A CLASSIC EXAMPLE

- Given a sorted list, it repeatedly divides the search space in half.
- With each comparison, the problem size is halved.

If you start with n elements:

- After 1 step: $n/2$
- After 2 steps: $n/4$
- ...
- After k steps: $n/2^k$

The search ends when $n/2^k = 1$, so $2^k = n \rightarrow k = \log_2 n$

Time complexity: $O(\log n)$

Implication: Even for very large n (e.g. 1 billion), $\log(10^9) = 30$ steps suffice.

COMMON GROWTH RATES (ORDERED BY EFFICIENCY)

Complexity	Name	Example Algorithms
$O(1)$	Constant	Accessing array element
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort, Quick Sort (average)
$O(n^2)$	Quadratic	Bubble sort
$O(2^n)$	Exponential	Naive Fibonacci, subset generation
$O(n!)$	Factorial	Traveling Salesman (brute force)

Logarithmic and Linearithmic algorithm scale well; exponential ones do not

PRACTICAL TIPS

- **Logarithmic time** is excellent: even with huge datasets, execution remains fast.
 - **Exponential time** is a red flag: consider optimization or approximation.
 - When you see “halving” or “tree with branching factor 2”, think **logarithms**.
 - When you see “trying all combinations” or “recursive branching”, think **exponentials**.
-

Summary

- **Exponents** model explosive growth — common in brute-force algorithms.
- **Logarithms** model slow, manageable growth — hallmark of efficient algorithms.
- Together, they help us understand and classify algorithm efficiency.
- Choosing an $O(\log n)$ algorithm over $O(2^n)$ can mean the difference between instant results and waiting centuries.

ALGORITHMIC NOTATION

1. Identifying Number
2. Steps, Control, Exit
3. Comments
4. Variable Names
5. Assignment Statement
6. Input and Output
7. Procedures

CONTROL STRUCTURES

1. Sequence logic or sequential flow
2. Selectic logic or conditional flow
3. Iteration logic or repetitive flow

A formal presentation of this algorithm, whose flow chart appears in Fig. 2.2, follows.

Algorithm 2.1:

(Largest Element in Array) A nonempty array DATA with N numerical values is given. This algorithm finds the location LOC and the value MAX of the largest element of DATA. The variable K is used as a counter.

- Step 1. [Initialize.] Set $K := 1$, $LOC := 1$ and $MAX := DATA[1]$.
- Step 2. [Increment counter.] Set $K := K + 1$.
- Step 3. [Test counter.] If $K > N$, then:
Write: LOC, MAX, and Exit.
- Step 4. [Compare and update.] If $MAX < DATA[K]$, then:
Set $LOC := K$ and $MAX := DATA[K]$.
- Step 5. [Repeat loop.] Go to Step 2.

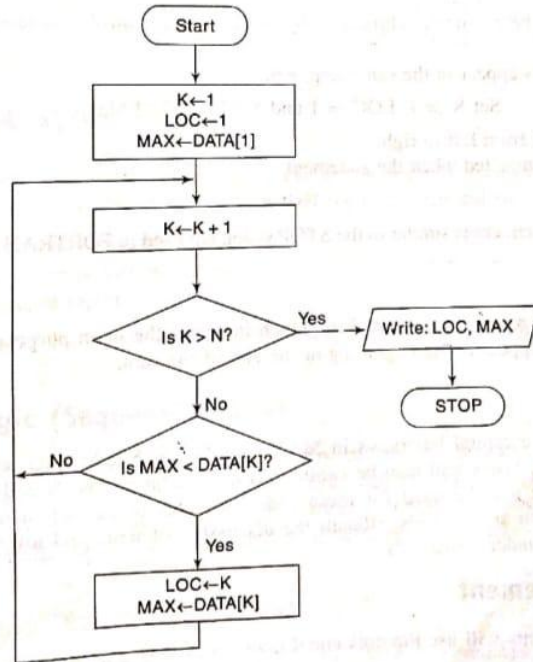


Fig. 2.2

USING REPEAT-WHILE LOOP

Algorithm 2.1 is rewritten using a repeat-while loop rather than a Go to statement:

Algorithm 2.3: (Largest Element in Array) Given a nonempty array DATA with N numerical values, this algorithm finds the location LOC and the value MAX of the largest element of DATA.

1. [Initialize.] Set $K := 1$, $LOC := 1$ and $MAX := DATA[1]$.
2. Repeat Steps 3 and 4 while $K \leq N$:
3. If $MAX < DATA[K]$, then:
 Set $LOC := K$ and $MAX := DATA[K]$.
 [End of If structure.]
4. Set $K := K + 1$.
 [End of Step 2 loop.]
5. Write: LOC, MAX.
6. Exit.

ASYMPTOTIC NOTATION

When analyzing algorithms, we want to:

- Compare their efficiency, especially in terms of time and space.
- Ignore machine-dependent constants and lower-order terms.
- Focus on input size (n) and how it affects performance as n grows large.

This leads to Asymptotic Analysis, which examines the behavior of algorithms as input size $\rightarrow \infty$.

BASIC TERMINOLOGIES

Term	Description
Input Size(n)	The size of the input data (e.g., number of elements in an array)
Time complexity	Number of basic operations or steps algorithm takes
Space Complexity	Amount of extra memory used by the algorithm

WHAT IS ASYMPTOTIC NOTATION?

Asymptotic notation provides a mathematical way to describe the upper, lower, and tight bounds of an algorithm's complexity.

There are 3 primary notations:

1. Big-0 notation - worst-case upper bound
2. Big-Omega notation - best-case lower bound
3. Theta notation - tight-bound average case

1. Big-O Notation (O) – *Worst-case upper bound*

It gives an upper bound on the time. The algorithm **won't take more** than this time (in the worst case).

Definition:

A function $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

Example:

If $f(n) = 3n + 2$, then $f(n) = O(n)$

2. Omega Notation (Ω) – *Best-case lower bound*

It provides a **guarantee** that the algorithm takes **at least this much time** (in the best case).

Definition:

$f(n)$ is $\Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that:

$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0$$

 **Example:**

If $f(n) = 3n + 2$, then $f(n) = \Omega(n)$

3. Theta Notation (Θ) – *Tight bound (average-case)*

It gives both the upper and lower bounds, i.e., the exact growth rate.

Definition:

$f(n)$ is $\Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and $n_0 \geq 0$ such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0$$

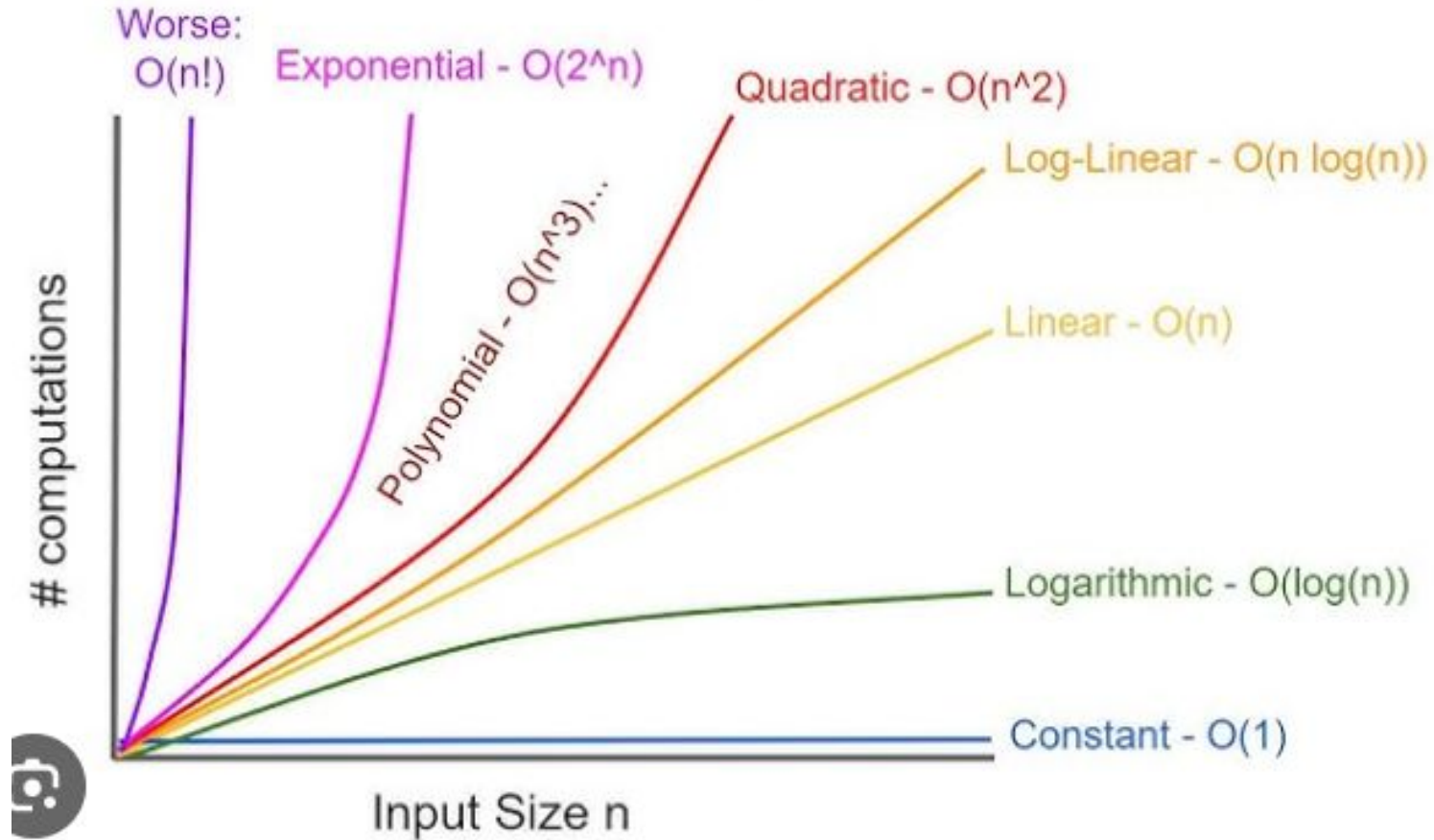
 **Example:**

If $f(n) = 3n + 2$, then $f(n) = \Theta(n)$

COMMON ASYMPTOTIC COMPLEXITIES

Notation	Name	Example Algorithm
$O(1)$	Constant time	Accessing an array element
$O(\log n)$	Logarithmic time	Binary search
$O(n)$	Linear time	Linear search
$O(n \log n)$	Log-linear time	Merge sort, Quick sort (avg)
$O(n^2)$	Quadratic time	Bubble sort, Selection sort
$O(2^n)$	Exponential time	Recursive Fibonacci
$O(n!)$	Factorial time	Traveling Salesman (Brute force)

COMMON ASYMPTOTIC COMPLEXITIES



- An algorithm is said to have **constant time with order 0 (1)** when it is not dependent on the input size n . Irrespective of the input size n , the runtime will always be the same.
- An algorithm is said to have a **linear time complexity** when the running time **increases linearly** with the length of the input. When the function involves checking all the values in input data, with this order **$O(n)$** .

- An algorithm is said to have a **logarithmic time complexity** when it **reduces** the size of the input data in each step.
- This indicates that the number of operations is not the same as the input size. **The number of operations gets reduced as the input size increases.**
- Algorithms are found in **binary trees or binary search functions**. This involves the search of a given value in an array by **splitting the array** into two and starting searching in one split. This ensures the operation is not done on every element of the data.

- An algorithm is said to have a non-linear time complexity where the running time increases **non-linearly (n^2)** with the length of the input.
- Generally, **nested loops** come under this order where one loop takes $O(n)$ and if the function involves a loop within a loop, then it goes for $O(n)*O(n) = O(n^2)$ order.
-
- Similarly, if there are 'm' loops defined in the function, then the order is given by **$O(n^m)$** , which are called **polynomial time complexity functions**.

HOW TO COMPUTE BIG-O FROM A GIVEN PROBLEM OR CODE

Step 1: Identify the Input Size (n)

Figure out what n represents. Common examples:

- Length of an array
- Number of elements in a list
- Number of nodes in a tree
- Size of a string

HOW TO COMPUTE BIG-O FROM A GIVEN PROBLEM OR CODE

Step 2: Look for Loops

Every loop often contributes to the complexity.

Loop Pattern	Complexity
Single <code>for</code> loop	$O(n)$
Nested <code>for</code> loop	$O(n^2)$, $O(n^3)$, etc.
Loop that halves each time	$O(\log n)$

HOW TO COMPUTE BIG-O FROM A GIVEN PROBLEM OR CODE

Step 3: Count nested structures

```
for i in range(n):           # 0(n)
    for j in range(n):       # 0(n)
        print(i, j)          # Total: 0(n × n) = 0(n2)
```

If you see one loop
inside another,
multiply their
complexities:

HOW TO COMPUTE BIG-O FROM A GIVEN PROBLEM OR CODE

Step 4: Analyze Recursion (if present)

Use recurrence relations or recursion trees to compute time.

Step 5: Ignore Constants and Lower Order Terms

- **Drop constants:**

$$O(2n) \rightarrow O(n)$$

$$O(100n + 50) \rightarrow O(n)$$

- **Drop lower-order terms:**

$$O(n^2 + n) \rightarrow O(n^2)$$

HOW TO COMPUTE BIG-O FROM A GIVEN PROBLEM OR CODE

Step 6: Add or Multiply Based on Control Flow

Sequential operations → Add complexities

$$O(n) + O(\log n) = O(n)$$

Nested operations → Multiply complexities

$$O(n) * O(\log n) = O(n \log n)$$

HOW TO COMPUTE BIG-O FROM A GIVEN PROBLEM OR CODE

Step 7: Use Big-O rules

Pattern	Result
Constant time operation	$O(1)$
Loop from 1 to n	$O(n)$
Loop inside a loop (both n)	$O(n^2)$
Recursive halving	$O(\log n)$
Loop + loop (not nested)	$O(n + n) = O(n)$
Loop with inner log ops	$O(n \log n)$

For example, to calculate the big-O notation for

$$f(n) = n \frac{(n+1)}{2} = \frac{1}{2} n^2 + \frac{1}{2} n$$

we first remove all coefficients. This gives us

$$n^2 + n$$

which after removing the smaller factors gives us

$$n^2$$

which in big-O notation is stated as

$$O(f(n)) = O(n^2)$$

To consider another example, let's look at the polynomial expression

$$f(n) = a_j n^k + a_{j-1} n^{k-1} + \dots + a_2 n^2 + a_1 n + a_0$$

We first eliminate all of the coefficients as shown below.

$$f(n) = n^k + n^{k-1} + \dots + n^2 + n + 1$$

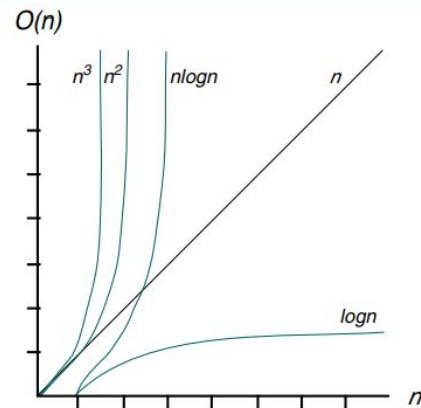
The largest term in this expression is the first one, so we can say that the order of a polynomial expression is

$$O(f(n)) = O(n^k)$$

STANDARD MEASURES OF EFFICIENCY

Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10,000^2$	minutes
Polynomial	$O(n^k)$	$10,000^k$	hours
Exponential	$O(c^n)$	$2^{10,000}$	intractable
Factorial	$O(n!)$	$10,000!$	intractable

Measures of Efficiency for $n = 10,000$



Plot of Efficiency Measures

ALGORITHM EFFICIENCY

- Linear Loops

-

Linear Loops

Let us start with a simple loop. We want to know how many times the body of the loop is repeated in the following code:⁴

```
for (i = 0; i < 1000; i++)  
    application code
```

Assuming i is an integer, the answer is 1000 times. The number of iterations is directly proportional to the loop factor, 1000. The higher the factor, the higher the number of loops. Because the efficiency is directly proportional to the number of iterations, it is

$$f(n) = n$$

However, the answer is not always as straightforward as it is in the above example. For instance, consider the following loop. How many times is the body repeated in this loop? Here the answer is 500 times. Why?

```
for (i = 0; i < 1000; i += 2)  
    application code
```

In this example the number of iterations is half the loop factor. Once again, however, the higher the factor, the higher the number of loops. The efficiency of this loop is proportional to half the factor, which makes it

$$f(n) = n / 2$$

If you were to plot either of these loop examples, you would get a straight line. For that reason they are known as **linear loops**.

ALGORITHM EFFICIENCY

- **Logarithmic Loops**

Logarithmic Loops

In a linear loop, the loop update either adds or subtracts. In a **logarithmic loop**, the controlling variable is multiplied or divided in each iteration. How many times is the body of the loops repeated in the following program segments?

Multiply Loops

```
for (i = 0; i < 1000; i *= 2)  
    application code
```

Divide Loops

```
for (i = 0; i < 1000; i /= 2)  
    application code
```

Multiply		Divide	
Iteration	Value of i	Iteration	Value of i
1	1	1	1000
2	2	2	500
3	4	3	250
4	8	4	125
5	16	5	62
6	32	6	31
7	64	7	15
8	128	8	7
9	256	9	3
10	512	10	1
(exit)	1024	(exit)	0

Analysis of Multiply and Divide Loops

Generalizing the analysis, we can say that the iterations in loops that multiply or divide are determined by the following formula:

$$f(n) = \log n$$

ALGORITHM EFFICIENCY

- ## Nested Loops

Loops that contain loops are known as **nested loops**. When we analyze nested loops, we must determine how many iterations each loop completes. The total is then the product of the number of iterations in the inner loop and the number of iterations in the outer loop.

```
Iterations = outer loop iterations x inner loop iterations
```

We now look at three nested loops: linear logarithmic, quadratic, and dependent quadratic.

Linear Logarithmic

The inner loop in the following code is a loop that multiplies. To see the multiply loop, look at the update expression in the inner loop.

```
for (i = 0; i < 10; i++)  
    for (j = 0; j < 10; j *= 2)  
        application code
```

The number of iterations in the inner loop is therefore $\lceil \log_{10} \rceil$. However, because the inner loop is controlled by an outer loop, the above formula must be multiplied by the number of times the outer loop executes, which is 10. This gives us

$$10 \log_{10}$$

which is generalized as

$$f(n) = n \log n$$

Quadratic

In a **quadratic loop**, the number of times the inner loop executes is the same as the outer loop. Consider the following example.

```
for (i = 0; i < 10; i++)  
    for (j = 0; j < 10; j++)  
        application code
```

The outer loop (*for i*) is executed 10 times. For each of its iterations, the inner loop (*for j*) is also executed 10 times. The answer, therefore, is 100, which is 10×10 , the square of the loops. This formula generalizes to

$$f(n) = n^2$$

Dependent Quadratic

In a **dependent quadratic loop**, the number of iterations of the inner loop depends on the outer loop. Consider the nested loop shown in the following example.

```
for (i = 0; i < 10; i++)  
    for (j = 0; j < i; j++)  
        application code
```

The outer loop is the same as the previous loop. However, the inner loop depends on the outer loop for one of its factors. It is executed only once the first iteration, twice the second iteration, three times the third iteration, and so forth. The number of iterations in the body of the inner loop is calculated as shown below.

$$1 + 2 + 3 + \dots + 9 + 10 = 55$$

If we compute the average of this loop, it is 5.5 (55/10), which is the same as the number of iterations (10) plus 1 divided by 2. Mathematically, this calculation is generalized to

$$\frac{(n + 1)}{2}$$

Multiplying the inner loop by the number of times the outer loop is executed gives us the following formula for a dependent quadratic loop:

$$f(n) = n\left(\frac{n + 1}{2}\right)$$

BIG-O ANALYSIS EXAMPLES

Add Square Matrices

To add two square matrices, we add their corresponding elements; that is, we add the first element of the first matrix to the first element of the second matrix, the second element of the first matrix to the second element of the second matrix, and so forth. Of course, the two matrices must be the same size. This concept is shown in Figure 1-15.

4	2	1
0	-3	4
5	6	2

 +

6	1	7
3	2	-1
4	6	2

 =

10	3	8
3	-1	3
9	12	4

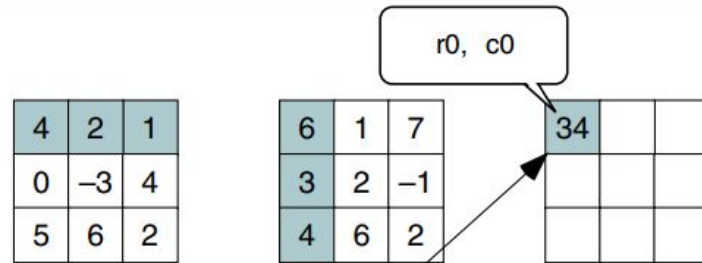
Add Matrices

Add Two Matrices

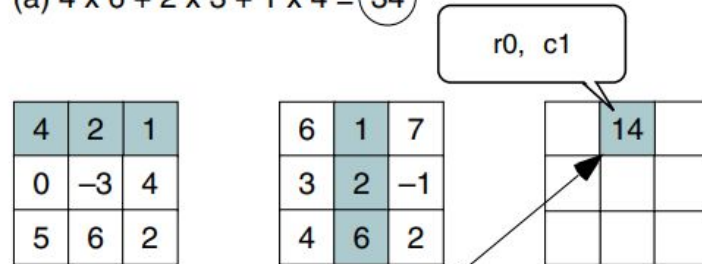
```
Algorithm addMatrix (matrix1, matrix2, size, matrix3)
Add matrix1 to matrix2 and place results in matrix3
  Pre  matrix1 and matrix2 have data
       size is number of columns or rows in matrix
  Post matrices added--result in matrix3
1 loop (not end of row)
  1 loop (not end of column)
    1 add matrix1 and matrix2 cells
    2 store sum in matrix3
  2 end loop
2 end loop
end addMatrix
```

In this algorithm, we see that for each element in a row, we add all of the elements in a column. This is the classic quadratic loop. The efficiency of the algorithm is **$O(n^2)$**

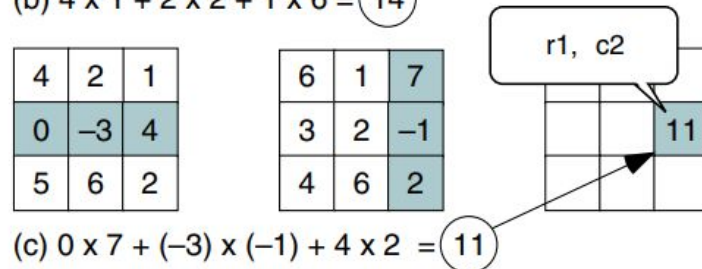
MATRIX MULTIPLICATION



(a) $4 \times 6 + 2 \times 3 + 1 \times 4 = 34$



(b) $4 \times 1 + 2 \times 2 + 1 \times 6 = 14$



(c) $0 \times 7 + (-3) \times (-1) + 4 \times 2 = 11$

Multiply Two Matrices

```
Algorithm multiMatrix (matrix1, matrix2, size, matrix3)
Multiply matrix1 by matrix2 and place product in matrix3
  Pre  matrix1 and matrix2 have data
       size is number of columns and rows in matrix
  Post matrices multiplied--result in matrix3
1 loop (not end of row)
  1 loop (not end of column)
    1 loop (size of row times)
      1 calculate sum of
        (all row cells) * (all column cells)
      2 store sum in matrix3
```

continued

Multiply Two Matrices (*continued*)

```
    2 end loop
  2 end loop
3 return
end multiMatrix
```

In this algorithm we see **three nested loops**. Because each loop starts at the first element, we have a cubic loop. Loops with three nested loops have a big-O efficiency of **$O(n^3)$**

COMMON PROPERTIES OF ASYMPTOTIC NOTATIONS

◆ 1. Reflexivity

A function is always in its own class.

✳ Property:

- $f(n) \in O(f(n))$
- $f(n) \in \Omega(f(n))$
- $f(n) \in \Theta(f(n))$

✓ Example:

Let $f(n) = 3n + 2$,

Then,

- $f(n) \in O(3n + 2)$
- $f(n) \in \Omega(3n + 2)$
- $f(n) \in \Theta(3n + 2)$

COMMON PROPERTIES OF ASYMPTOTIC NOTATIONS

◆ 2. Transitivity

If $f(n)$ is in the class of $g(n)$, and $g(n)$ is in the class of $h(n)$, then $f(n)$ is in the class of $h(n)$.

✳ Property:

- If $f(n) \in O(g(n))$ and $g(n) \in O(h(n)) \Rightarrow f(n) \in O(h(n))$
- Similarly applies to $\Omega()$ and $\Theta()$.

✓ Example:

- $f(n) = n$
- $g(n) = 3n$
- $h(n) = 5n + 7$

Then:

$$n \in O(3n), 3n \in O(5n + 7) \Rightarrow n \in O(5n + 7)$$



COMMON PROPERTIES OF ASYMPTOTIC NOTATIONS

◆ 3. Symmetry (*Only for Θ*)

Theta is symmetric. If $f(n)$ tightly bounds $g(n)$, then the reverse is also true.

✳ Property:

- $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$

✓ Example:

- $f(n) = 2n + 1$
- $g(n) = n$

Since $f(n)$ and $g(n)$ grow at the same rate,

$$\Rightarrow f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$$

◆ 4. Transpose Symmetry (*Between O and Ω*)

This shows the inverse relationship between O and Ω .

✳ Property:

- $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$

✓ Example:

If $f(n) = n$, $g(n) = n^2$,

Then: $n \in O(n^2) \Leftrightarrow n^2 \in \Omega(n)$

◆ 5. Additivity (Sum Rule)

When adding two functions of different growth rates, the **dominant** one determines the complexity.

✱ Property:

- $O(f(n) + g(n)) = O(\max(f(n), g(n)))$

✓ Example:

- If $f(n) = n$, $g(n) = n^2$,

Then $O(f(n) + g(n)) = O(n + n^2) = O(n^2)$

The **larger term dominates**.

◆ 6. Multiplicativity (Product Rule)

When multiplying two functions, the total complexity is the product of their complexities.

✳ Property:

- If $f(n) \in O(h(n))$ and $g(n) \in O(k(n))$,
Then $f(n) \cdot g(n) \in O(h(n) \cdot k(n))$

✓ Example:

If $f(n) = n$, $g(n) = \log n$

Then: $f(n) \cdot g(n) = n \log n \in O(n \log n)$

◆ 7. Polynomial Bound Rule

For a polynomial $f(n) = a_0 + a_1n + a_2n^2 + \dots + a_kn^k$,

The highest-degree term dominates.

✱ Property:

- $f(n) = \Theta(n^k)$

✓ Example:

If $f(n) = 5n^3 + 4n^2 + 7$,

Then $f(n) = \Theta(n^3)$

◆ 8. Logarithmic Rule

Logarithms of different bases are asymptotically equivalent.

✱ Property:

- $\log_a(n) \in \Theta(\log_b(n))$, for any $a, b > 1$

✓ Example:

- $\log_2(n) \in \Theta(\log_{10}(n))$

This is because:

$$\log_a(n) = \log_b(n) / \log_b(a) \Rightarrow \text{difference is just a constant.}$$

◆ 9. Non-negativity

Only non-negative functions can be used in asymptotic analysis.

✓ Example:

If $f(n) = -n$, we cannot write $f(n) \in O(n)$

Instead, define a new function $g(n) = |f(n)| = n$ and use it for asymptotics.

◆ 10. Composition Rule

If $f(n) = O(g(n))$, then $f(h(n)) = O(g(h(n)))$ if $h(n)$ is increasing.

✓ Example:

Let $f(n) = n$, $g(n) = n^2$

If $f(n) \in O(g(n)) \Rightarrow f(\log n) \in O(g(\log n)) \Rightarrow \log n \in O((\log n)^2)$

Property	Applies To	Description
Reflexivity	All	A function belongs to its own class
Transitivity	All	Relationships carry over transitively
Symmetry	Θ only	Mutual tight bound
Transpose Symmetry	O & Ω	Flip relationship
Additivity	All	Dominant term survives
Multiplicativity	All	Complexities multiply
Polynomial Bound	Θ	Highest term dominates

Logarithmic Rule

Θ

Logs with different bases are equivalent

Composition Rule

All

Composition preserves order

Non-negativity

All

Only use functions ≥ 0

The next topic is Unit 2- Recursion